

I implemented a prompting and reasoning module that bases on the datasets that produced by the preprocessing pipeline. The prompting system is implemented in "prompt.py" and is organized around reusable prompt templates and a unified execution interface. Each prompt method uses a formatted instruction that includes the problem text while enforcing a standardized answer format. This constraint simplifies future evaluation because model outputs can be automatically parsed to extract the final answer. All prompting strategies share a common system instruction that encourages careful reasoning and ensures consistent behavior across different experimental conditions.

Five prompting strategies are supported in the current implementation. The Direct Prompt serves as a baseline where the model is asked to solve the problem without explicit reasoning instructions. Chain-of-Thought (CoT) prompting encourages the model produce step-by-step reasoning before generating the final answer, which can improves performance on multi-step problems. Least-to-Most prompting decomposes complex tasks into smaller subproblems and solves them sequentially, encouraging the model build reasoning process. Self-Consistency improves robustness by sampling multiple CoT reasoning paths and selecting the most frequent answer through majority voting. Finally, Plan-and-Solve prompting introduces an explicit planning stage.

All prompting methods are used through a unified interface (`run_prompt_method`), which includes the model, tokenizer, question text, and the selected prompting strategy as inputs. This abstraction allows the experiment runner to switch between prompting methods while keeping the dataset and model configuration fixed. The design ensures that the entire project remains reproducible, extensible, and easy to analyze.